

Lab 4 - Vibe Coding: 2048 (Scenario 14)

LLM chat history · Claude Code (VS Code) · Pranav Kalla, PES1UG24AM197 · 08 Oct 2026

Source repo: github.com/SETAPESU26/14_2048 · Submission: SE_PES1UG24AM197_LAB/LAB_4

PROMPT

Read README.md, board.py, game.py and main.py. Explain briefly how a move works, from input to tile creation. Don't change anything yet.

CLAUDE

Read 4 files (ctrl+o to expand)

CLAUDE

Here's the flow for one move, from keypress to new tile:

1. main.py (LAB_4/main.py) just calls Game().run().
2. game.py:27-47 (LAB_4/game.py#L27-L47) – run() loops: displays the board, checks win (2048 in any row) and loss (can_move() false), then reads a key via input(). For q/u it short-circuits; for w/a/s/d it calls self.move(key).
3. game.py:17-25 (LAB_4/game.py#L17-L25) – move(key) maps the key to one of board.move_left/right/up/down, calls it, and gets back changed (bool). If changed is True, it calls self.board.add_random_tile() to spawn a new tile. It returns changed so run() can update best_score.
4. board.py:30-64 (LAB_4/board.py#L30-L64) – each move_* method extracts each row (or column, reversed for right/down) as a line, saves old, runs it through slide_line, writes the result back into self.grid, and ORs into changed whether that line differed from old.
5. board.py:20-28 (LAB_4/board.py#L20-L28) – slide_line(line) does the actual compression/merge: drops zeros (values), then walks values left-to-right, merging into the previous slot via result[-1] *= 2 when equal, else appending. Pads with zeros to SIZE and returns. Note the comment: this is the intentional double-merge bug (Task 1) – e.g. [2,2,2,2] merges into 4, then that newly-created 4 merges again with the next 4, collapsing to a single 8 instead of the standard [4,4,0,0], because result[-1] can be re-merged against on the same pass.
6. board.py:66-75 (LAB_4/board.py#L66-L75) – can_move() checks for any empty cell or adjacent equal pair, used by run() to detect game-over.

Back in game.py, once add_random_tile() fires, a 2 (90%) or 4 (10%) is placed in a random empty cell – completing the move cycle. Note also: there's currently no undo (u just prints a message), no distinct win/loss state beyond printing+return, and move() doesn't yet guard against spawning a tile when the move didn't actually change the board in all cases – these are what Tasks 2-4 ask you to address.

PROMPT

Task 1: Fix slide_line so each tile merges at most once per move ([2,2,2,2] should become [4,4,0,0], not 8), and make the score increase by merged values.

CLAUDE

Update(board.py)

```
└─ Added 22 lines, removed 9 lines
20     def slide_line(line):
21         values = [x for x in line if x]
22         result = []
23         for value in values:
24             if result and result[-1] == value:
25                 result[-1] *= 2 # intentional double-merge bug
23 +         gained = 0
```

```

24 +         i = 0
25 +         while i < len(values):
26 +             if i + 1 < len(values) and values[i] == values[i +
+1]:
27 +                 merged = values[i] * 2
28 +                 result.append(merged)
29 +                 gained += merged
30 +                 i += 2
31 +             else:
27 -                 result.append(value)
28 -         return result + [0] * (SIZE - len(result))
32 +                 result.append(values[i])
33 +                 i += 1
34 +         return result + [0] * (SIZE - len(result)), gained
35
36     def move_left(self):
37         changed = False
38         for r in range(SIZE):
39             old = self.grid[r][:]
34 -             self.grid[r] = self.slide_line(old)
40 +             new, gained = self.slide_line(old)
41 +             self.grid[r] = new
42 +             self.score += gained
43             changed |= old != self.grid[r]
44         return changed
45
...
47         changed = False
48         for r in range(SIZE):
49             old = self.grid[r][:]
42 -             self.grid[r] = list(reversed(self.slide_line(list(r
-reversed(old))))))
50 +             new, gained = self.slide_line(list(reversed(old)))
51 +             self.grid[r] = list(reversed(new))
52 +             self.score += gained
53             changed |= old != self.grid[r]
54         return changed
55
...
57         changed = False
58         for c in range(SIZE):
59             old = [self.grid[r][c] for r in range(SIZE)]
50 -             new = self.slide_line(old)
60 +             new, gained = self.slide_line(old)
61             for r in range(SIZE):
62                 self.grid[r][c] = new[r]
63 +             self.score += gained
64             changed |= old != new
65         return changed
66
...
68         changed = False
69         for c in range(SIZE):
70             old = [self.grid[r][c] for r in range(SIZE)]
60 -             new = list(reversed(self.slide_line(list(reversed(o
-ld))))))
71 +             new, gained = self.slide_line(list(reversed(old)))
72 +             new = list(reversed(new))
73             for r in range(SIZE):
74                 self.grid[r][c] = new[r]
75 +             self.score += gained
76             changed |= old != new
77         return changed
78

```

Ran 1 shell command (ctrl+o to expand)

CLAUDE

[2,2,2,2] now slides to [4,4,0,0] with score +8, confirming each tile merges at most once. Fixed in board.py:19-33 (LAB_4/board.py#L19-L33) (slide_line now walks with an index step of 2 on a merge instead of re-merging into result[-1]) and board.py:35-74 (LAB_4/board.py#L35-L74) (each move_* now unpacks (new, gained) and adds gained to self.score).

PROMPT

Task 2: Make reaching 2048 end the game, a full board with no moves end the game, and unchanged moves never add a tile. Also reject invalid input like an empty Enter.

CLAUDE

Read 1 file, ran 1 shell command (ctrl+o to expand)

CLAUDE

Update(game.py)

```
| Added 1 line, removed 1 line
40         if key == "u":
41             print("Undo is not implemented yet.")
42             continue
43 -         if key not in "wasd":
43 +         if key not in ("w", "a", "s", "d"):
44             print("Use W/A/S/D.")
45             continue
46         if self.move(key):
```

Ran 2 shell commands (ctrl+o to expand)

CLAUDE

Confirmed the fix and that the other two requirements were already correct:

- Empty Enter / invalid input: the old check key not in "wasd" used Python's substring semantics, where "" in "wasd" is True – so pressing Enter silently slipped past the validity check and fell through to move(""), which no-ops but prints nothing. Fixed at game.py:43 (LAB_4/game.py#L43) by checking membership in the tuple ("w","a","s","d") instead of the string, which correctly rejects "" and any other non-single-key input with the "Use W/A/S/D." message.
- Reaching 2048 ends the game: already correct – game.py:31-33 (LAB_4/game.py#L31-L33) checks 2048 in row each loop and returns.
- Full board, no moves ends the game: already correct – game.py:34-36 (LAB_4/game.py#L34-L36) uses board.can_move().
- Unchanged move creates no tile: already correct – game.py:22-24 (LAB_4/game.py#L22-L24) only calls add_random_tile() when changed is True, verified above with a pre-slid row.

So the only real defect for Task 2 was the empty-input check; it's now fixed.

PROMPT

Task 3: Add one-level undo with U that restores board and score without adding a tile. Keep best score in memory only, with no files.

CLAUDE

Read 1 file (ctrl+o to expand)

CLAUDE

Update(game.py)

```
| Added 2 lines
```

```

1 +import copy
2 +
3   from board import Board
4
5

```

CLAUDE

Update(game.py)

└ Added 10 lines

```

21             "w": self.board.move_up, "s": self.board.move_
            down}
22         if key not in moves:
23             return False
24 +         snapshot = (copy.deepcopy(self.board.grid), self.board.
+score)
25         changed = moves[key]()
26         if changed:
27 +             self.history = [snapshot]
28             self.board.add_random_tile()
29         return changed
30
31 +     def undo(self):
32 +         if not self.history:
33 +             return False
34 +         grid, score = self.history.pop()
35 +         self.board.grid = grid
36 +         self.board.score = score
37 +         return True
38 +
39     def run(self):
40         print("2048 – W/A/S/D to move, U to undo, Q to quit.")
41         while True:

```

CLAUDE

Update(game.py)

└ Added 2 lines, removed 1 line

```

50             if key == "q":
51                 return
52             if key == "u":
53 -                 print("Undo is not implemented yet.")
53 +                 if not self.undo():
54 +                     print("Nothing to undo.")
55                 continue
56             if key not in ("w", "a", "s", "d"):
57                 print("Use W/A/S/D.")

```

CLAUDE

Now let me verify the behavior: undo restores grid+score, doesn't add a tile, and can't be chained twice.

Ran 1 shell command (ctrl+o to expand)

CLAUDE

Confirmed: undo exactly reverses one move (board and score both restored), adds no tile, and a second consecutive u is rejected since it's one-level only.

Changes in game.py (LAB_4/game.py):

- move() now snapshots (deepcopy(grid), score) before attempting a move, and only commits it to self.history (replacing any prior entry – hence "one-level") when the move actually changed the board.
- New undo() pops that snapshot and restores self.board.grid/self.board.score directly – no call to add_random_tile(), so no new tile appears.

- The u branch in run() now calls self.undo() and prints "Nothing to undo." when there's no snapshot (either nothing moved yet, or undo was already used).

best_score is untouched by undo – it's a running max already kept only in memory (self.best_score), matching "no files" and the usual 2048 convention that your best score for the run doesn't drop just because you undid a move.

PROMPT

Task 4: Print one short feedback message per successful move, and say "no move" for unchanged or invalid moves.

CLAUDE

Read 1 file (ctrl+o to expand)

CLAUDE

```
Update(game.py)
└─ Added 5 lines
    22         if key not in moves:
    23             return False
    24         snapshot = (copy.deepcopy(self.board.grid), self.board.
        score)
    25 +     score_before = self.board.score
    26         changed = moves[key]()
    27         if changed:
    28 +             gained = self.board.score - score_before
    29             self.history = [snapshot]
    30             self.board.add_random_tile()
    31 +             print(f"Merged! +{gained}" if gained else "Moved.")
    32 +         else:
    33 +             print("No move.")
    34             return changed
    35
    36     def undo(self):
```

CLAUDE

```
Update(game.py)
└─ Added 1 line, removed 1 line
    59             print("Nothing to undo.")
    60             continue
    61         if key not in ("w", "a", "s", "d"):
    62 -             print("Use W/A/S/D.")
    62 +             print("No move. Use W/A/S/D.")
    63             continue
    64         if self.move(key):
    65             self.best_score = max(self.best_score, self.boa
        rd.score)
```

CLAUDE

Let me verify the feedback prints exactly once per case and that can_move() scans don't trigger anything.

Ran 1 shell command (ctrl+o to expand)

CLAUDE

All cases confirmed:

- Successful move with a merge → "Merged! +8" (shows the score gained).
- Successful move with no merge → "Moved."
- Unchanged move (valid direction, nothing shifted) → "No move."
- Invalid key → "No move. Use W/A/S/D."

- `can_move()` (called every loop just to check game-over) prints nothing – feedback only fires from inside `move()` on an actual key press, never from internal scanning.

Changes are in `game.py:19-29` (`LAB_4/game.py#L19-L29`) (`move()` now diffs score before/after to pick the message and prints exactly once) and `game.py:56-58` (`LAB_4/game.py#L56-L58`) (invalid-key message reworded to lead with "No move.").